

To ChAI or Not to ChAI:

Chapel+AI for Training and Inference

Mohamed Abdelnaby, Paul Sathre, and Wu-chun Feng
{mohamedabdelnaby, psath6220, wfeng}@vt.edu
Department of Computer Science, Virginia Tech, USA

30th IEEE High Performance Extreme Computing Conference (HPEC)

VIRGINIA TECH

SyNeRG
synergy.cs.vt.edu

Motivation





	Python	Chapel
Performance & Efficiency	✗ (interpreted)	✓ (compiled)
Productivity (Parallel / Distributed)	✓ / ○	✓ / ✓
Scalability (Parallel / Distributed)	✓ / ✗	✓ / ✓
Deep Learning	✓	✗

- Compiled HPC languages (Fortran, C/C++, Chapel) ***cannot*** natively train neural networks.
- Python frameworks with deep learning (e.g., PyTorch) ***cannot*** easily scale to the large (e.g., million nodes).

Efficiency

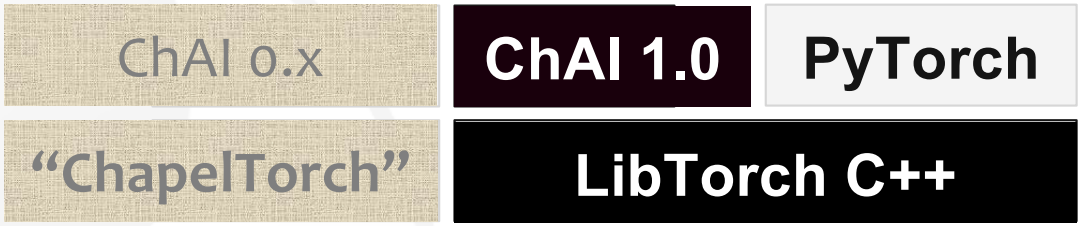
	Energy
(c) C	1.00
(c) Rust	1.03
(c) C++	1.34
(c) Ada	1.70
(v) Java	1.98
(c) Pascal	2.14
(c) Chapel	2.18
(v) Lisp	2.27
(c) Ocaml	2.40
(c) Fortran	2.52
(c) Swift	2.79
(c) Haskell	3.10
(v) C#	3.14
(c) Go	3.23
(i) Dart	3.83
(v) F#	4.13
(i) JavaScript	4.45
(v) Racket	7.91
(i) TypeScript	21.50
(i) Hack	24.02
(i) PHP	29.30
(v) Erlang	42.23
(i) Lua	45.98
(i) Jruby	46.54
(i) Ruby	69.91
(i) Python	75.88
(i) Perl	79.58

Contributions

- ChAI 1.0: Chapel + AI for Training and Inference
  - Performance, productivity, scalability, and deep learning
  - A compiled front-end that lowers host cost for every operation
- Performance evaluation of ChAI 1.0 vs. PyTorch

	PyTorch	ChAI 1.0
Performance & Efficiency	✗ (interpreted)	✓ (compiled)
Productivity (Parallel / Distributed)	✓ / ○	✓ / ✓
Scalability (Parallel / Distributed)	✓ / ✗	✓ / ✓ (soon)
Deep Learning	✓	✓

Approach: Bringing AI to Chapel via LibTorch C++



Past Work (Inference Only)

- ChAI 0.x: Re-implement AI kernels in Chapel ("ChapelTorch")
  - Effort: Reproduce years of cuBLAS / cuDNN tuning
  - Result: Trails PyTorch on the layers that dominate AI

This Work (Training & Inference)

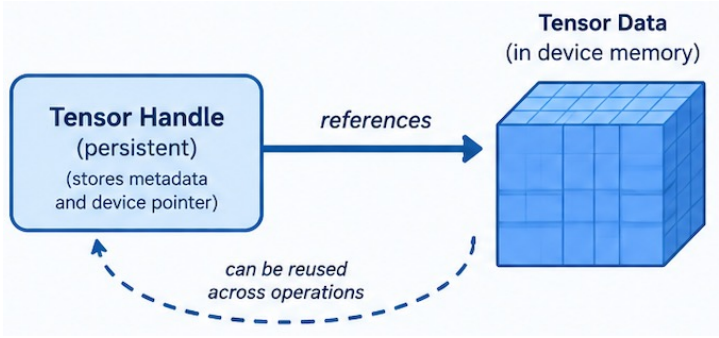
- ChAI 1.0: Bind to LibTorch C++
  - Effort: Leverage the same AI kernels that PyTorch does
  - Result: Chapel front-end (for ChAI 1.0) outperforms Python front-end (for PyTorch)

Capability	ChAI 0.x	ChAI 1.0	PyTorch
Inference	✓	✓	✓
Training (autograd, optimizers)	✗	✓	✓
Persistent tensor handles	✗	✓	✓
Optimizers (Adam, SGD, fused)	✗	✓	✓
Loss functions	◐	✓	✓
CNN and vision layers	✓	✓	✓
Transformer and attention	✗	✓	✓
Recurrent (LSTM, GRU)	✗	✓	✓
CPU and GPU device API	✗	✓	✓
CUDA Graphs	✗	✓	✓
Operator fusion	✗	✗	✓
Mixed precision (BF16, FP8)	✗	◐	✓
Load PyTorch checkpoints	✓	✓	✓
Tensor backend	Chapel	LibTorch	LibTorch

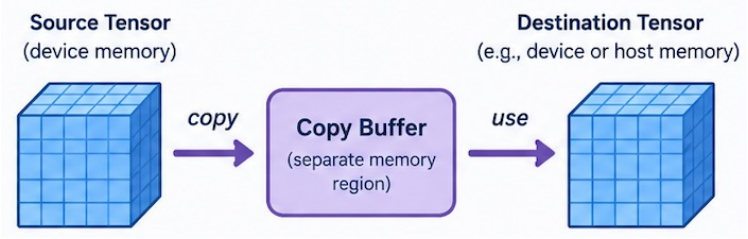
✓ supported ◐ partial or experimental ✗ absent

Design of ChAI 1.0: Chapel+AI for Training and Inference

- **Persistent tensor handles** → reusable reference (no data movement)
  - Operations pass pointers instead of copying data
- **Training "switched on"**
  - LibTorch builds the *backward graph* during the forward pass
- **Every operator dispatched to LibTorch**
- **Device control and graph capture**
  - New tensors inherit their inputs' device
  - GPU request never silently falls back to the CPU
- **AI Execution Mode: Eager → Graphs**

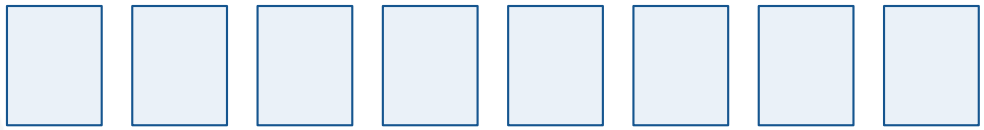


vs.



AI Execution Mode: Eager vs. ChAI

- Eager
  - One launch *per* operation



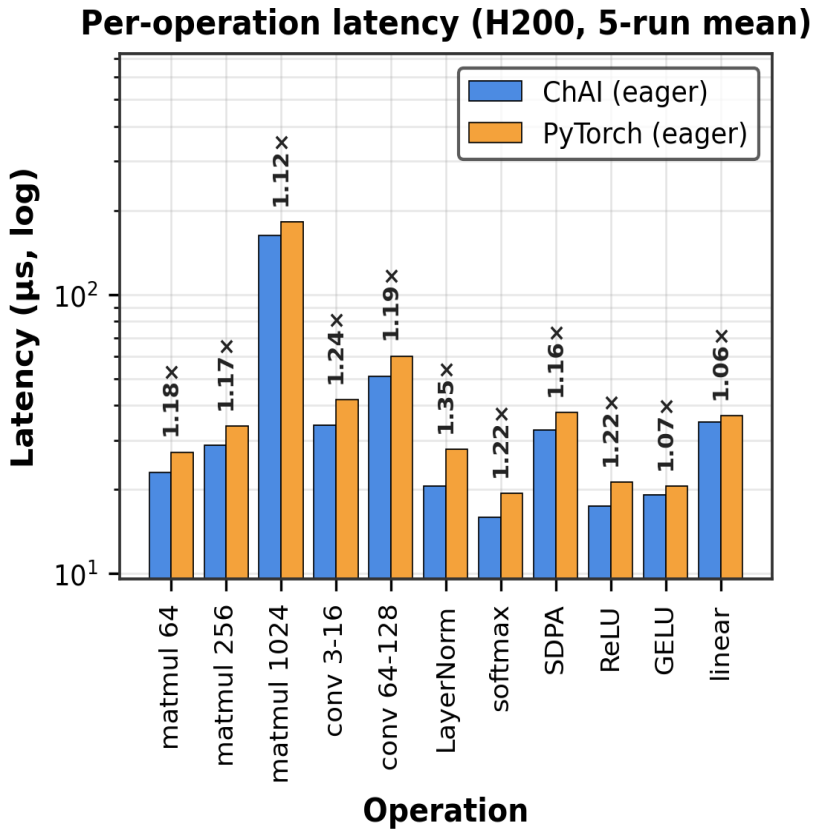
8 Ops → 8 Launches

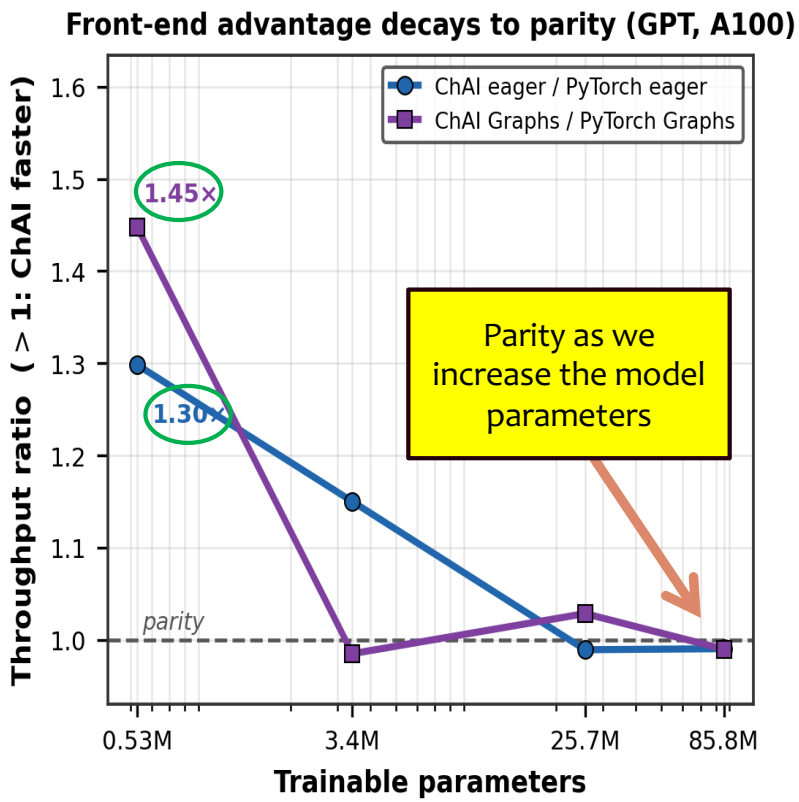
- Graphs
  - Capture once, replay as one graph launch

One graph launch

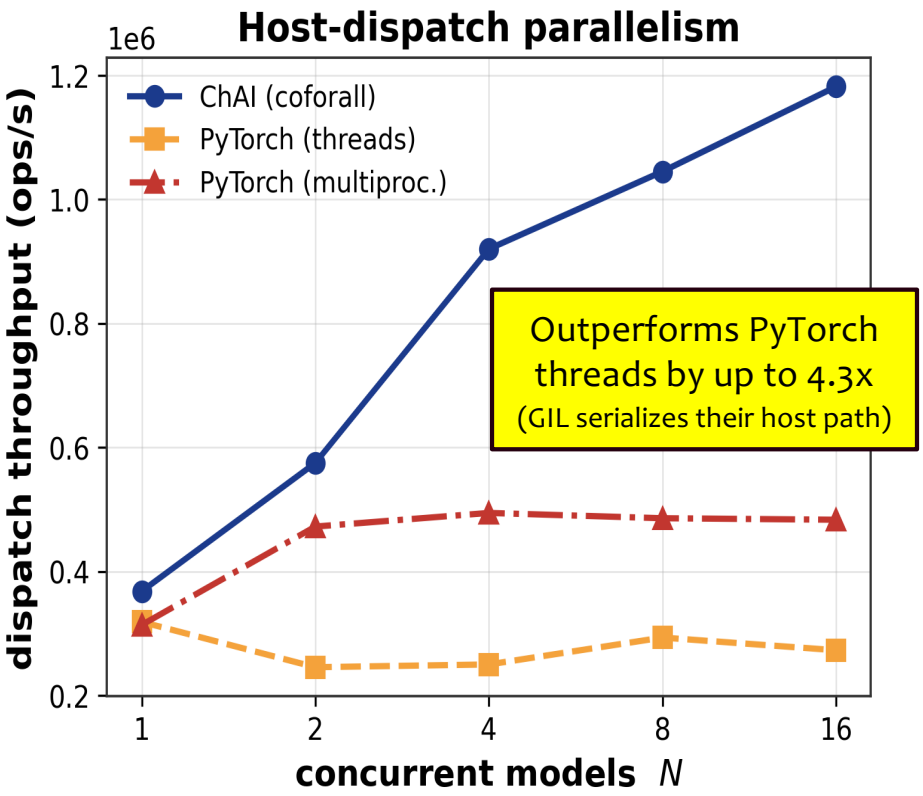
Same kernels → One launch

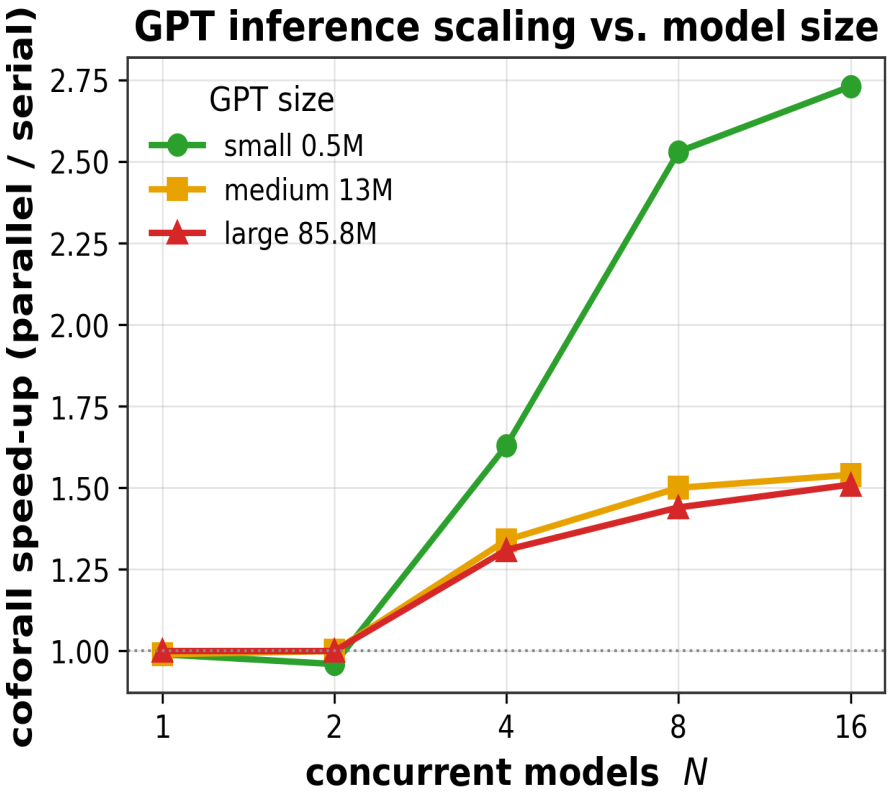
Training Results: Per-operation & Throughput





Inference Results: Parallelism





Future Work: ChAI *Distributed* GPU AI Training

- ChAI 1.0 (and PyTorch) deliver ***scalable parallel training*** *within* a single node …

	PyTorch	ChAI 1.0
Performance & Efficiency	✗ (interpreted)	✓ (compiled)
Productivity (Parallel / Distributed)	✓ /	✓ / ✓
Scalability (Parallel / Distributed)	✓ / ✗	✓ / ✓ (SOON)
Deep Learning	✓	✓

Chapel, as a parallel & distributed language …

- Can naturally deliver ***scalable distributed training*** to ChAI 1.0 across multiple nodes
- Enable exploration of Chapel's GPU-aware communication for gradient synchronization across GPUs

Conclusion

- ChAI 1.0
  - Chapel+AI for training and inference
    - Via a compiled Chapel interface that removes measurable model-size-dependent host overhead
    - Trains at PyTorch throughput with no kernel rewrites
    - Can scale many models per process
    - Delivers in-language parallelism for inference that PyTorch alone *cannot* match

Thank you for your attention. Questions?



synergy.cs.vt.edu